



WRF AND PANGU WEATHER DEMO

ALL TUTORIAL MATERIALS ARE AVAILABLE AT [HTTPS://GITHUB.COM/CONGGIAN/WRF_TRAINING_SCIPE/TREE/MAIN](https://github.com/conggian/wrf_training_scipe/tree/main)

PANGU WEATHER MODEL

- 265 million parameter model trained on 5th generation of ECMWF reanalysis (ERA5) data.
- Global simulations at 0.25 degree resolution
- 5 surface variables (Z, Q, T, U, and V)
- 15 vertical levels (1000hPa, 925hPa, 850hPa, 700hPa, 600hPa, 500hPa, 400hPa, 300hPa, 250hPa, 200hPa, 150hPa, 100hPa, and 50hPa)
- Fully data-driven training process.
- Official Github Repository (<https://github.com/I98808xc/Pangu-Weather>)

AI MODEL CONDA AND DIRECTORY SETUP

- Any conda module (Conda, Anaconda, or Miniconda) can be used and would be the only system requirement to run this demonstration.
- The final Conda environment contains all packages necessary to run the AI models and carry out the necessary data postprocessing for the AI and WRF output.
 - Conda packages can be substituted for system packages, especially cuda-toolkit and cudnn (neural networks package)
- Cfgrib is installed for working with ERA5 grib2 data.
- Ipykernel is installed for use of Conda environment in Jupyter notebook sessions if needed.
 - The kernel is installed at the end of the script

```
git clone https://github.com/CongGian/WRF_training_SCIPE.git
cd WRF_training_SCIPE/pangu_example

## Set variables and create directories for conda and python package to not flood
## directory
#export SCRATCH=/anvil/scratch/$USER/
#export CONDA_ENVS_PATH=$SCRATCH/conda/envs
#export CONDA_PKGS_DIRS=$SCRATCH/conda/pkgs
#export PIP_CACHE_DIR=$SCRATCH/pip/cache

## Load Modules and create env
ml conda/2025.09

conda create -n pangu python=3.11
conda activate pangu

# CuDNN package for neural networks and cuda-toolkit
# Must be compatible with each other and onnx library in requirements_gpu.txt
conda install -c nvidia cuda-toolkit=12.9.1 cudnn
# CuDNN is not of the system

# GPU
pip install -r requirements_gpu.txt

## Install ipykernel if wanted to use Jupyter notebook for anything
conda install -c conda-forge cartopy
conda install -c conda-forge cfgrib
conda install -c conda-forge ipykernel

# Install ipykernel for jupyter notebook use
python -m ipykernel install --name pangu --display-name "Python (Pangu)" --user
```

MODEL DEMO WORKFLOW

- Untarring the data file produces the “forecasts/2011-04-27-00-00” directory
`tar -xjvf data_input.tar.xz`
- Following the “run_workflow.sh” file, create a “models” directory, and move your downloaded model weights into the directory.
- For this example, the Python scripts have been edited to the correct dates for you.
- Additionally, to produce a run with different data:
 - Refer to `anvil_setup.sh` for directions for generating a CDS api key
 - Refer to item #2 in the `run_workflow.sh` script.

MODEL INFERENCE

- The inference.py can be set to run on GPUs
 - Defaults to CPU if no GPU is detected.
- Script selects the most efficient combination of model weights to carry out inference.
 - 36-hour forecast > 24h, 6hr, 6hr (To the right)
 - 24-hour forecast > 24hr
 - 40-hour forecast > 24hr, 6hr, 6hr, 3hr, 1hr
- When the GPU option is enabled, and no GPU is detected, the script will default to CPU
- However, this mechanism can lead to the program being killed.

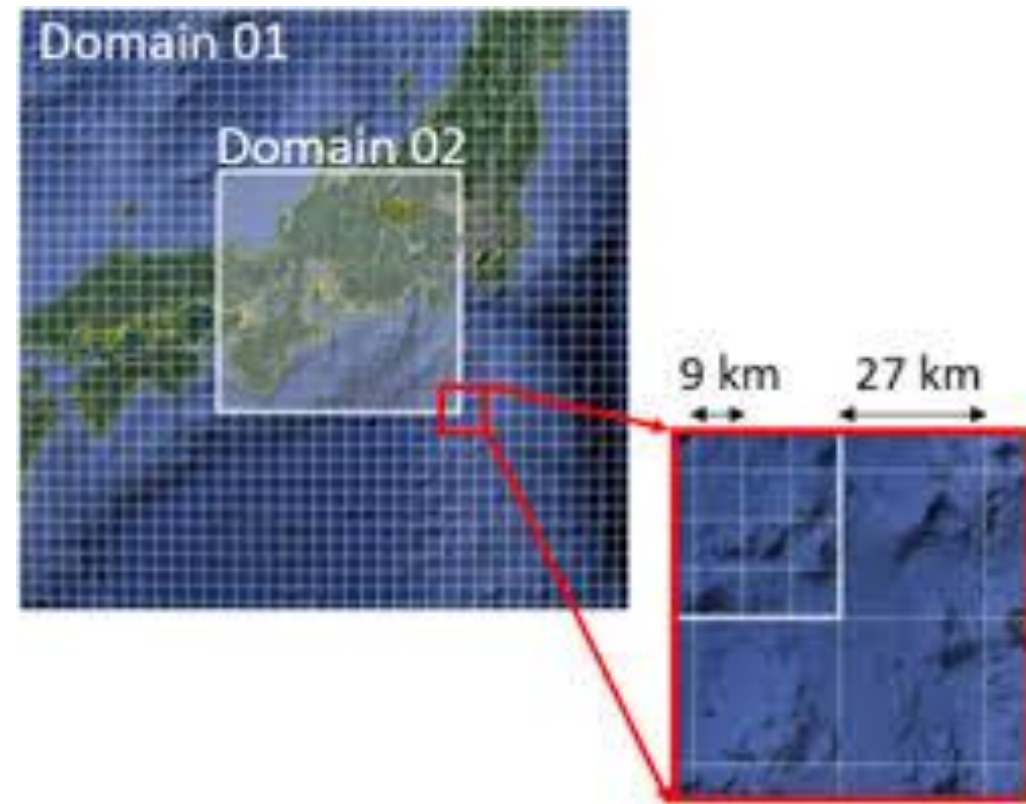
```
36.0
24
2011-04-28-00-00
start
12.0
6
2011-04-28-06-00
start
6.0
6
2011-04-28-12-00
start
```

MODEL OUTPUT PROCESSING

- The “inference.py” script creates a “results/<date>” directory that contains the “surface” and “upper” output in numpy data files. E.g.:
 - output_surface_2011-04-28-00-00.npy
 - output_upper_2011-04-28-00-00.npy
 - An output file is produced for each model run (24-hr, 6-hr, 6-hr)
- The “forecast_decode.py” script converts these output files to netcdf files and stores them in the “outputs” folder that is created. E.g.:
 - output_surface_2011-04-28-00-00.nc
 - output_upper_2011-04-28-00-00.nc
 - Each of the output files is converted to a netcdf file

WRF-ARW

- <https://www.mmm.ucar.edu/models/wrf>
- https://www2.mmm.ucar.edu/wrf/users/wrf_users_guide/build/html/overview.html
- We will be using WRF-ARW (Advanced Research WRF)
- Limited area model
 - Simulations are carried out in a specified model domain, requiring the conditions at the edge of this domain to be updated periodically.
- Optional nesting
 - Parent domain has a course resolution than the inner domain.
 - Allows for fine-resolution simulations while avoiding directly interpolating very coarse input data onto a fine resolution grid.
- Offers a large range of physics parameterizations
 - Parameterization options let the model know how to represent or quantify the sub-grid-scale processes because they are too small to simulate directly.



WEATHER RESEARCH AND FORECASTING MODEL (WRF) DEMO

■ Workflow

■ WPS (WRF Preprocessing System)

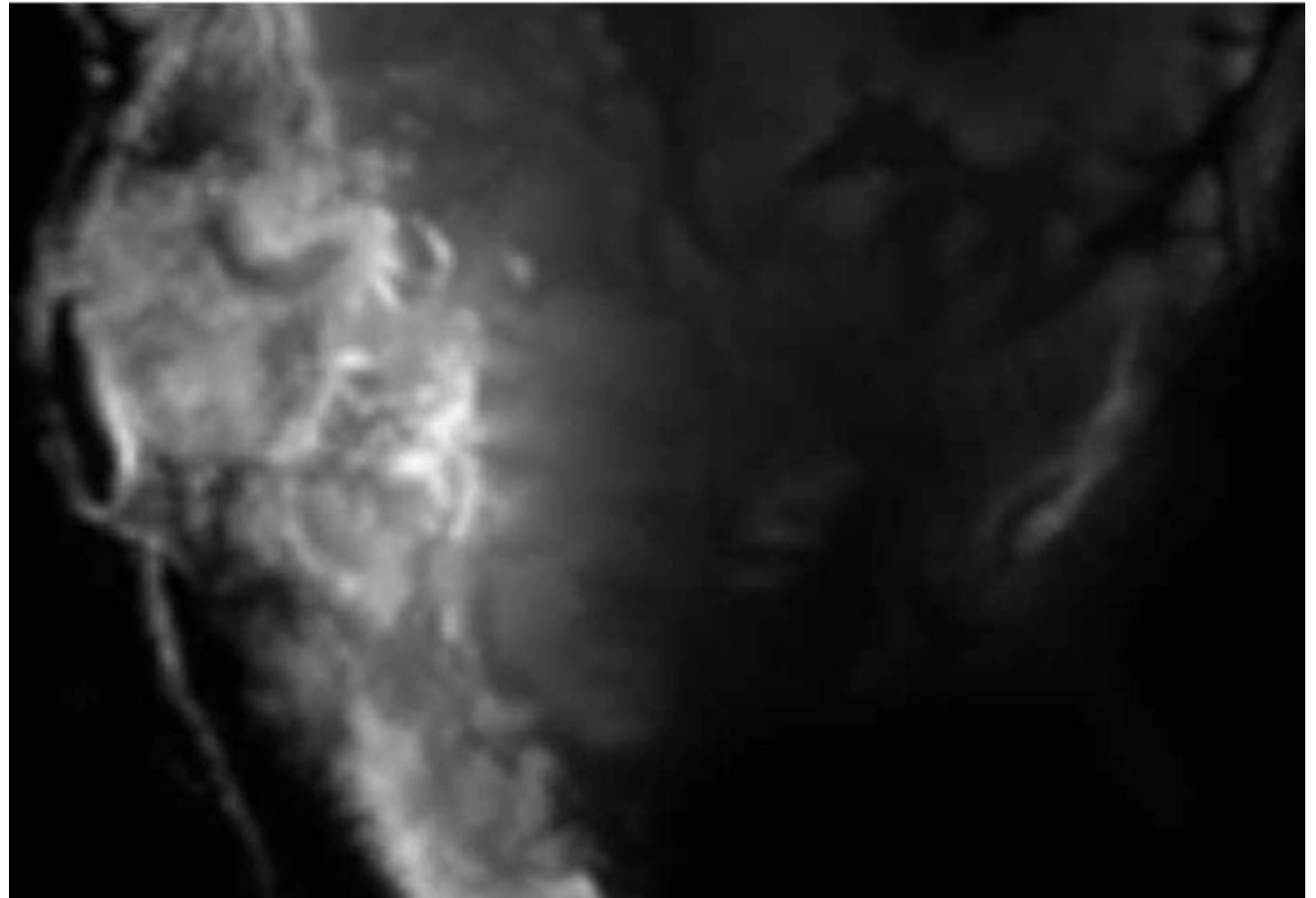
- Geogrid.exe: static geography interpolation onto the domain, producing a “geo_em*” for each domain
- Ungrib.exe: Produces intermediate meteorology file from input data (e.g., ERA5, NARR, GFS, etc.)
- Metgrid.exe: Interpolates met data horizontally to the domain using the geography file, producing “met_em*” files

■ WRF

- Real.exe: Interpolates met data vertically, producing both input (wrfinput) and boundary (wrfbdy) conditions files for the simulation
- Wrf.exe: Carries out model integration, producing the final output of the WRF simulation

WPS

- WRF/WPS compilation and execution instructions:
 - https://www2.mmm.ucar.edu/wrf/OnLineTutorial/compilation_tutorial.php
- WPS has been fully run, and the “geo_em.d01.nc” file is in the GitHub repo
 - This eliminates the need to download the WPS geography data, but the intermediate meteorology files and “met_em*” files must be created.



RUNNING METGRID.EXE USING CONTAINER

- Utilizes 10 cores
- Loads Anvil compiler and MPI compatible with those in the container.
- “Apptainer exec” executes metgrid.exe using the container.
- “—mpi=pmi2” allows for effective communication between cluster and container MPI.

```
#!/bin/bash

#!/bin/sh -l

#SBATCH -A cis240917
#SBATCH -p shared # the default queue is "shared" queue
#SBATCH --nodes=1
#SBATCH --ntasks=10
#SBATCH --mem=20G
#SBATCH --time=1:00:00
#SBATCH --job-name wps
#SBATCH --mail-user=ak1817@msstate.edu # e-mail address
#SBATCH --mail-type=BEGIN
#SBATCH --mail-type=END
#SBATCH --mail-type=FAIL

# Unload all modules
ml purge

# Load mvapich2 for parallel execution
ml gcc
ml mvapich2

export SCRATCH=/anvil/scratch/$USER

# Run program using container
srun --mpi=pmi2 apptainer exec $SCRATCH/wps_wrf/wrf.sif $SCRATCH/test/WPS/metgrid.exe
```

RUNNING METGRID.EXE USING CONTAINER

- Real.exe can run in serial
 - Delete output files to rerun to avoid a fatal error
- To use OpenMP to execute wrf.exe simply at the “—cpus-per-task” option
 - This is create multiple threads for each process and assigning then each to a single core

```
#!/bin/sh -l

#SBATCH -A cis240917
#SBATCH -p shared # the default queue is "shared" queue
#SBATCH --nodes=1
#SBATCH --ntasks=10
##SBATCH --ntasks=20 # Uncomment when not using openmp to maintain 20 cores
#SBATCH --cpus-per-task=2 # Uncomment when not using openmp to maintain 20 cores
#SBATCH --time=1:00:00
#SBATCH --job-name wrf
#SBATCH --mail-user=ak1817@msstate.edu # e-mail address
#SBATCH --mail-type=BEGIN
#SBATCH --mail-type=END
#SBATCH --mail-type=FAIL

# Unload all modules
ml purge

# Load mvapich2 for parallel execution
ml gcc
ml mvapich2

# Link met_em files to wrf run directory
ln -s ../../WPS/met_em* .

export SCRATCH=/anvil/scratch/$USER

# Run real.exe to get input and boundary conditions files
apptainer exec $SCRATCH/wps_wrf/wrf.sif $SCRATCH/test/WRF/run/real.exe

# Run wrf program using container
srun --mpi=pmi2 apptainer exec $SCRATCH/wps_wrf/wrf.sif $SCRATCH/test/WRF/run/wrf.exe
```

WRF TIMESTEPS AND OUTPUT

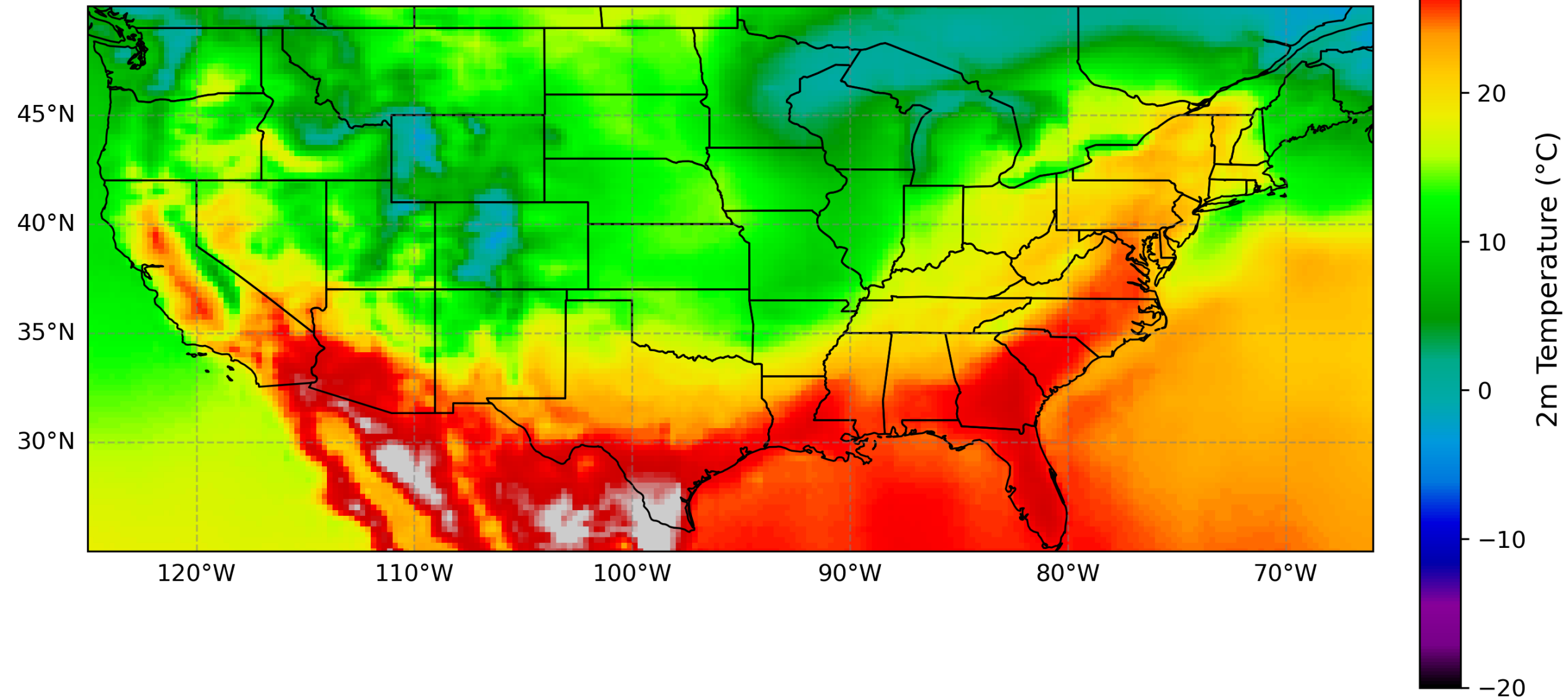
- Wrf.exe logs all timesteps in the rsl.out* files
- Simulation speed can be monitored using time step length
`tail -f rsl.out.0000`
- Wrf.exe outputs file at user-specified intervals, e.g.:
 - wrfout_d01_2011-04-28_00:00:00

```
Input data is acceptable to use: wrfbdy_d01
Timing for processing lateral boundary for domain 1: 0.10673 elapsed seconds
Timing for main: time 2011-04-27_03:01:30 on domain 1: 2.97031 elapsed seconds
Timing for main: time 2011-04-27_03:03:00 on domain 1: 0.29050 elapsed seconds
Timing for main: time 2011-04-27_03:04:30 on domain 1: 0.27460 elapsed seconds
Timing for main: time 2011-04-27_03:06:00 on domain 1: 0.28216 elapsed seconds
Timing for main: time 2011-04-27_03:07:30 on domain 1: 0.27469 elapsed seconds
Timing for main: time 2011-04-27_03:09:00 on domain 1: 0.28024 elapsed seconds
Timing for main: time 2011-04-27_03:10:30 on domain 1: 0.27208 elapsed seconds
Timing for main: time 2011-04-27_03:12:00 on domain 1: 0.27711 elapsed seconds
Timing for main: time 2011-04-27_03:13:30 on domain 1: 0.27959 elapsed seconds
Timing for main: time 2011-04-27_03:15:00 on domain 1: 0.27914 elapsed seconds
Timing for main: time 2011-04-27_03:16:30 on domain 1: 0.82021 elapsed seconds
Timing for main: time 2011-04-27_03:18:00 on domain 1: 0.28203 elapsed seconds
Timing for main: time 2011-04-27_03:19:30 on domain 1: 0.28365 elapsed seconds
Timing for main: time 2011-04-27_03:21:00 on domain 1: 0.27714 elapsed seconds
Timing for main: time 2011-04-27_03:22:30 on domain 1: 0.27701 elapsed seconds
Timing for main: time 2011-04-27_03:24:00 on domain 1: 0.27928 elapsed seconds
Timing for main: time 2011-04-27_03:25:30 on domain 1: 0.27608 elapsed seconds
Timing for main: time 2011-04-27_03:27:00 on domain 1: 0.27820 elapsed seconds
Timing for main: time 2011-04-27_03:28:30 on domain 1: 0.27612 elapsed seconds
Timing for main: time 2011-04-27_03:30:00 on domain 1: 0.27592 elapsed seconds
Timing for main: time 2011-04-27_03:31:30 on domain 1: 0.81463 elapsed seconds
Timing for main: time 2011-04-27_03:33:00 on domain 1: 0.28049 elapsed seconds
Timing for main: time 2011-04-27_03:34:30 on domain 1: 0.28247 elapsed seconds
Timing for main: time 2011-04-27_03:36:00 on domain 1: 0.27657 elapsed seconds
Timing for main: time 2011-04-27_03:37:30 on domain 1: 0.27911 elapsed seconds
Timing for main: time 2011-04-27_03:39:00 on domain 1: 0.27528 elapsed seconds
Timing for main: time 2011-04-27_03:40:30 on domain 1: 0.27609 elapsed seconds
Timing for main: time 2011-04-27_03:42:00 on domain 1: 0.27485 elapsed seconds
Timing for main: time 2011-04-27_03:43:30 on domain 1: 0.28470 elapsed seconds
Timing for main: time 2011-04-27_03:45:00 on domain 1: 0.28301 elapsed seconds
Timing for main: time 2011-04-27_03:46:30 on domain 1: 0.81284 elapsed seconds
Timing for main: time 2011-04-27_03:48:00 on domain 1: 0.38236 elapsed seconds
Timing for main: time 2011-04-27_03:49:30 on domain 1: 0.27357 elapsed seconds
Timing for main: time 2011-04-27_03:51:00 on domain 1: 0.27549 elapsed seconds
```

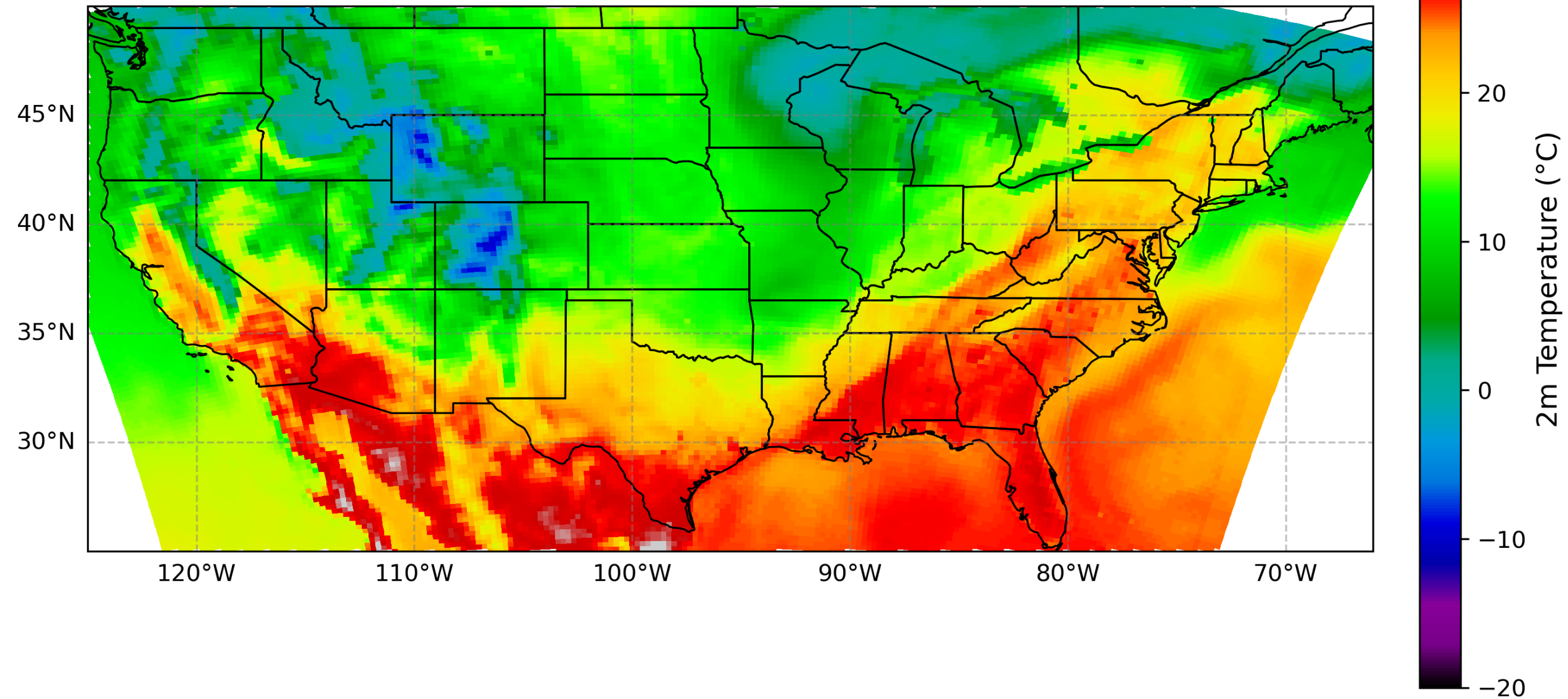
OUTPUT VISUALIZATION/COMPARISON

- The Python scripts and Jupyter notebooks are currently edited for April 28th, 2011 at 00z.
 - Day of the second-worst tornado outbreak in history.
- Each file outputs both a subset of the original data file (matching the domain) and plot 2-meter Temperature on the specified domain (0.25 degree spacing).
- ERA5 subset is already provided, so it is only necessary to run the first part of the script.

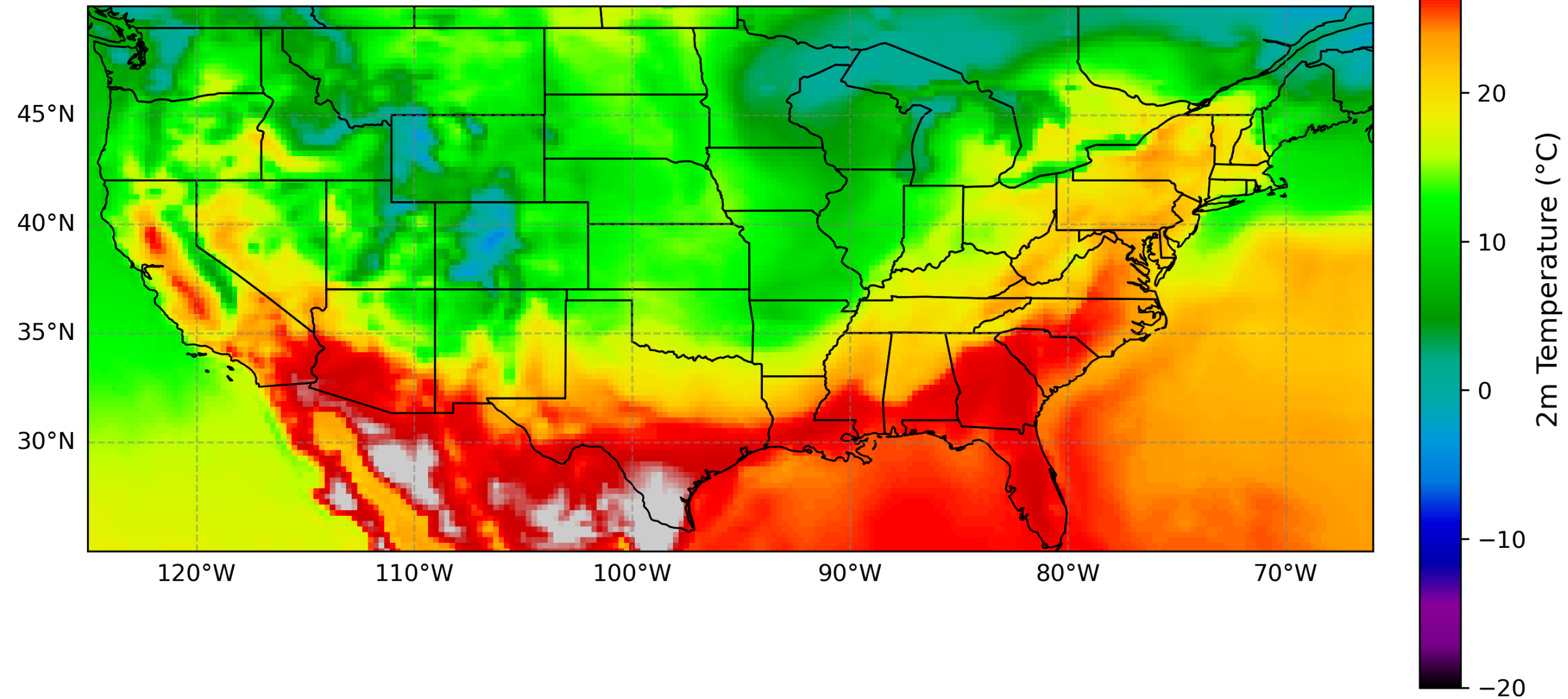
Pangu-Weather 2m Temperature: April 28, 2011 (00:00 UTC)



WRF Simulation 2m Temperature: April 28, 2011 (00:00 UTC)



ERA5 Reanalysis 2m Temperature: April 28, 2011 (00:00 UTC)



AI/DYNAMIC MODELS

AI

Pros

- Cost-efficient, running thousands of times faster than a dynamic model on a single GPU.
- Can produce consistent and relatively accurate predictions of variables such as temperature and wind.
- Decreased data storage requirements

Cons

- Very few output variables (Primarily T, Q, U/V)
- Non-customizable
- Limited vertical and horizontal resolution

Dynamic

Pros

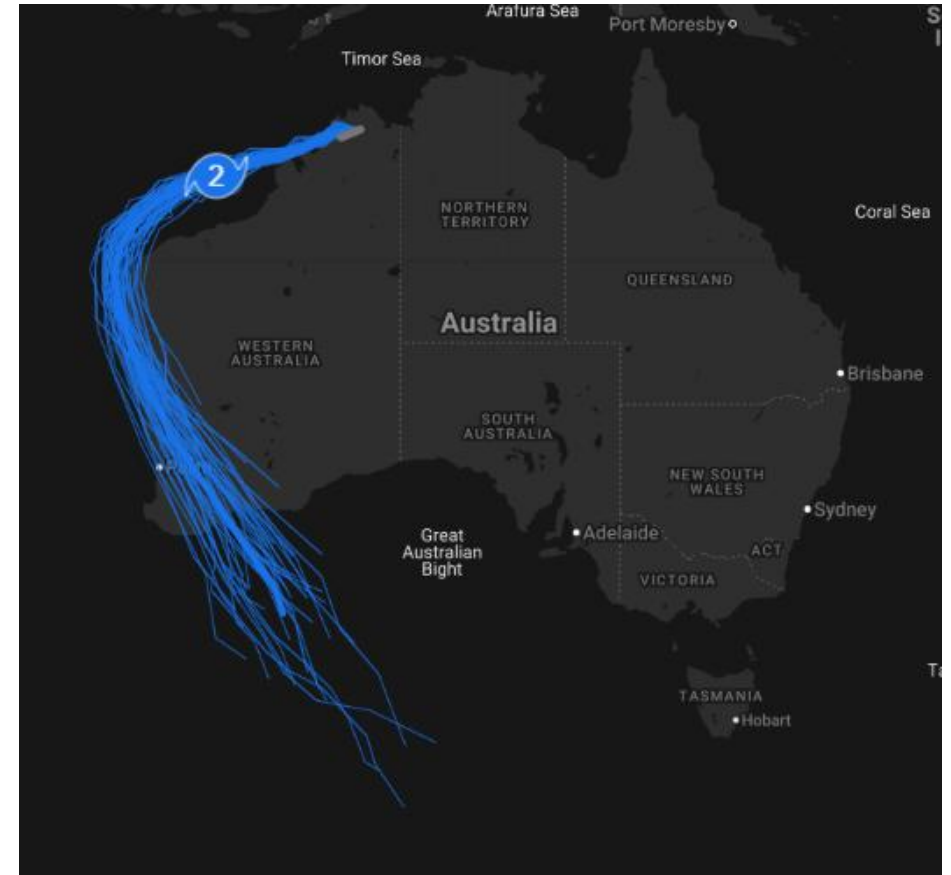
- > 100 output variables
- Adjustable horizontal and vertical resolution, including convective scales
- Customizable parameterizations

Cons

- Very cost-intensive, especially in fine-resolution simulations
- Requires dozens of GBs of data

AI/DYNAMIC MODELS

- Use cases
 - AI models serve primarily in operational contexts for forecasting
 - Google DeepMind AI hurricane forecasting ensemble model, forecasting hurricane track and intensity
 - ECMWF-AIFS for medium to long-range weather forecasting
 - Dynamic models serve a role in both operational and research contexts
 - Physically consistent simulations that can be used to study atmospheric processes.
 - WRF-ARW (Advanced Research WRF), used in the demo, is the most commonly used model for researching atmospheric phenomena.
 - Most models currently used by NOAA/NWS/SPC in forecasting are dynamic.



<https://deepmind.google.com/science/weatherlab>